Magpie — Sprint 10 Technical Spec (Sales Capture foundation)

Audience: implementing agent (Codex) + Regan. **Builds on** Sprints 0–9 (Phases 1–4 + hardening complete; runtime = Windows + SQLite, F-14). **Status: planning / review — implement after approval.**

> First sprint of Phase 6 (BI). Builds the **sale data foundation** the dashboards need: a `SaleRecord` source-of-record that captures what actually sold — including **partial-quantity sales** (Regan sells bulk lots in pieces) — and the **estimate-vs-realised** linkage that is the heart of the phase. **Manual capture only this sprint** (north-star: works with zero APIs). The model is built **sync-ready** so the next sprint's eBay order-sync populates the same table. **No dashboards yet** (that's the dashboards sprint), **no eBay sync** (its own sprint).

---

0. Locked decisions this spec implements

D-90 (Phase 6 = 3 sprints, manual-first foundation), D-91 (`SaleRecord` design), D-92 (partial-quantity selling + item quantity model + lifecycle + cost-basis allocation). Sync-readiness per D-93 (provenance field only — no sync here). Out of scope per D-95.

---

1. What gets built

1. `InventoryItem` **quantity model** — confirm/extend so items carry a total quantity; derive remaining from sales.
2. `SaleRecord` — provenance-tracked source-of-record for a sale (full or partial), linked to the item + its valuation, with actual + estimated-fee snapshots.
3. **Item lifecycle on sale** — `available` → `partially_sold` → `sold`, derived from quantity remaining, integrated with the existing status field.
4. **Cost-basis allocation** for partial sales (proportional default + per-sale override) → realised profit per sale.
5. **Manual sale-entry UI** + a basic **sales list/history**; **sales API**; backup coverage;

real-DB CI.

First: read the current `InventoryItem` model. It already has a status field (Sprint 1) and may model quantity via top-level fields or JSONB `attributes` (bulk lots). Integrate with what exists — do **not** create a parallel status/quantity system.

---

2. `InventoryItem` quantity (D-92)

- Ensure a `quantity_total` integer (default `1`; single items = 1; bulk lots set N). If the model already has a clean quantity, use it; otherwise add it (migration).
- **Do not store `quantity_remaining` as mutable state** — derive it: `quantity_sold = sum(SaleRecord.quantity for that item)`, `quantity_remaining = quantity_total - quantity_sold`. Deriving keeps it auditable and consistent (no decrement drift). Expose both as read-only computed fields/serializer values.
- Guard: a sale's quantity must be `≤ quantity_remaining` (validation, server-side).

---

3. `SaleRecord` model (D-91)

Source-of-record, provenance-tracked, one item → many sales. Fields:

- `item` (FK → InventoryItem, `related_name="sales"`)
- `sale_date` (date)
- `quantity` (PositiveInteger, default 1; **≤ remaining**)
- `sale_price` (Decimal — total for this sale's quantity, the gross the buyer paid)
- `channel` (`ebay_au` / `manual` / `other`)
- `actual_fees_total` (Decimal; manual entry **pre-filled** from the FeeSchedule estimate on `sale_price`, user-correctable)
- `actual_fee_breakdown` (JSON, optional — e.g. `{final_value, fixed, promoted}`)
- `actual_shipping_cost` (Decimal, default 0 — postage the seller bore)
- `net_proceeds` (Decimal, **derived**: `sale_price − actual_fees_total − actual_shipping_cost`)
- `allocated_cost_basis` (Decimal, **derived or overridden** — see §4)

- `realised_profit` (Decimal, **derived**: `net_proceeds − allocated_cost_basis`)
- `cost_basis_override` (Decimal, nullable — set for non-uniform lots; when null, proportional default applies)
- `listing_draft` (FK → ListingDraft, nullable — the draft/offer it sold through, if known)
- **`valuation_snapshot`** (JSON) — the item's *current* `ValuationReport` summary captured at sale time (estimated value, **per-unit** where the item is a lot), so historical est-vs-actual stays stable even if valuations change later
- **`estimated_fee_snapshot`** (JSON) — the FeeSchedule estimate at sale time (which schedule, the estimated fee on this sale_price), so est-vs-actual fees stays stable
- `provenance` (`manual` now; `ebay_sync` reserved for the sync sprint — **no sync logic here**)
- `corrected_from` (self-FK, nullable — corrections create a new row referencing the prior, never silent edits)
- `notes` (text, optional)
- `created_at` / `updated_at`

Derived fields computed in the model/serializer (not hand-entered). `SaleRecord` is **append/correct**, not freely mutable: a correction writes a new row pointing at `corrected_from` (mirrors the project's provenance discipline). Backup (`manage.py backup`) must include the new table.

---

4. Cost-basis allocation (D-92 — the partial-sale subtlety)

Realised profit needs the cost of the units in *this* sale. Two cases:

- **Uniform lot / single item (default — proportional):** `cost_per_unit = item.acquisition_cost / item.quantity_total`; `allocated_cost_basis = cost_per_unit × sale.quantity`. Automatic, no user input.
- **Non-uniform lot (override):** the user sets `cost_basis_override` for that sale (e.g. the one good coin in a junk lot carries most of the cost). When present, it wins.

(Use the item's existing acquisition-cost field — confirm its name in the model. If acquisition cost is absent/zero, `allocated_cost_basis` falls back to 0 with realised_profit still computed on net_proceeds, and the UI flags "no cost basis set.")

Keep it this simple in v1 — no FIFO/LIFO lot-layering.

---

5. Item lifecycle on sale (D-92)

Derived from quantity remaining, mapped onto the existing status field:

- `quantity_remaining == quantity_total` → **available** (or the existing pre-sale status)
- `0 < quantity_remaining < quantity_total` → **partially_sold** (new state — add to the status enum)
- `quantity_remaining == 0` → **sold**

Keep sold/partially-sold items visible and searchable (they are analytics data — never hard-delete). If the item had a live eBay listing, this sprint only *reflects* sold status locally (it does not call eBay — listing/offer reconciliation belongs to the sync sprint). Recompute status whenever a SaleRecord is created/corrected.

---

6. Manual sale-entry UI + sales list

- **"Record a sale"** action from the item detail (and optionally the inventory grid): fields = quantity (≤ remaining, with the remaining shown), sale_price (for that quantity), sale_date, channel, actual_fees_total (**pre-filled** from the FeeSchedule estimate on the entered price, editable), actual_shipping_cost, optional cost_basis_override, optional listing link, notes. Live-show the resulting net_proceeds + realised_profit before submit.
- **Sale history** on the item detail: this item's `SaleRecord`s, quantity remaining, total realised vs. item cost. (The *pretty* analytics are the dashboards sprint — here it's a clean functional list.)
- **Correction**: edit-as-new-row (writes `corrected_from`), not a silent overwrite.
- A simple global **sales list** view (recent sales) is acceptable but minimal.

---

7. API

DRF endpoints: create/list/retrieve `SaleRecord` (+ correction); item serializers expose `quantity_total`, `quantity_remaining`, `quantity_sold`, computed status, and nested/related sales. Server-side validation: quantity ≤ remaining; sane prices. No new

external calls.

---

8. Acceptance criteria

Back to a normal **real-DB CI gate** (this is feature code, fully testable):

1. `SaleRecord` model + migration; `manage.py backup` /restore include it.
2. **Partial sale** works: an item qty 10, sell 3 → remaining 7, status `partially_sold`; sell 7 more → remaining 0, status `sold`; a 4th sale is rejected (exceeds remaining).
3. **Cost-basis**: proportional default computes correctly (assert numbers, e.g. $100/10 × 3 = $30 → realised_profit = net − 30); override wins when set.
4. **Snapshots**: a SaleRecord captures the valuation + estimated-fee snapshots; changing the item's valuation/FeeSchedule afterward does **not** alter the stored snapshot (assert stability).
5. Derived fields (`net_proceeds` , `realised_profit` , `quantity_remaining`) compute correctly; correction creates a new row referencing `corrected_from` .
6. Manual sale-entry UI records a sale and updates item status + remaining; sold/partial items stay visible/searchable.
7. Frontend tests + typecheck + build green; backend tests green; **real-DB CI gate green** (dual SQLite+Postgres lanes, per the v4.x "Validation" workflow).

---

9. Out of scope (Sprint 10)

eBay order-sync / any eBay call (next sprint) · dashboards/charts/analytics aggregates (dashboards sprint) · FIFO/LIFO or lot-layer cost accounting · multi-currency · tax/accounting export · predictive/AI · automated listing-end on sale (sync sprint). **If you find yourself building charts or calling eBay, you've left the sprint.**

---

10. Seed

Optionally seed 1–2 example sales (incl. one partial-lot sale) so the next sprints have demo data — but **never** touch the live business data (F-14: the runtime DB is the real 8/2/5/2/1; seed only in dev/test/CI).

Hand-off notes

- **Derive `quantity_remaining` from SaleRecords; don't store a mutable counter** — avoids drift, stays auditable.
- **Snapshots are load-bearing**: without the valuation + estimated-fee snapshots, historical est-vs-actual breaks the moment a valuation or FeeSchedule changes.
- Integrate with the **existing** item status + acquisition-cost fields — read the model first; don't fork them.
- Provenance field is present so the sync sprint fills the same table — but **no sync code here**.
- Corrections are new rows (`corrected_from`), never silent edits — mirrors `ValuationReport` discipline.
- When ambiguous, prefer the smaller change that satisfies §8.